

Στέργιος Μουτζίκος inf2021149

Project_3 (Web Scraping for Data Collection)

Δεδομένα που χρησιμοποιήθηκαν για το Project 3(λινκς):

---> <https://en.wikipedia.org/wiki/Greece>

---> <https://www.bbc.com/weather>

---> <https://blog.hubspot.com/sales/famous-quotes>

---> <https://www.naftemporiki.gr/>

---> (Μπορούμε να εισάγουμε ότι λινκ ιστοσελίδας θέλουμε)

Εξήγηση του κώδικα :

---> 1.Εισαγωγή βιβλιοθηκών

```
1 import tkinter as tk #για δημιουργία γραφικών περιβαλλόντων
2 from tkinter import messagebox #για εμφάνιση μηνυμάτων
3 from tkinter import ttk #για προηγμένους ελέγχους εμφάνισης
4 import matplotlib.pyplot as plt # Εισαγωγή της βιβλιοθήκης matplotlib.pyplot για τη δημιουργία γραφικών
5 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg #για ενσωμάτωση γραφικών matplotlib σε παράθυρο tkinter
6 import requests #για αποστολή HTTP αιτημάτων
7 from bs4 import BeautifulSoup #για ανάλυση HTML
8 import csv #για εργασία με αρχεία CSV
9 from textstat import flesch_reading_ease #για υπολογισμό ευκολίας ανάγνωσης κειμένου
10
```

tkinter: Είναι μια βιβλιοθήκη που χρησιμοποιείται για τη δημιουργία γραφικών περιβαλλόντων στην Python. Επιτρέπει τη δημιουργία παραθύρων, κουμπιών, ετικετών και άλλων γραφικών στοιχείων.

messagebox: Αυτή η υπο-βιβλιοθήκη του tkinter χρησιμοποιείται για την εμφάνιση διαλόγων μηνυμάτων όπως πληροφορίες, προειδοποιήσεις και σφάλματα.

ttk: Είναι μια πρόσθετη βιβλιοθήκη του tkinter που παρέχει προηγμένους ελέγχους εμφάνισης, όπως κουμπιά με στυλ, μπάρες προόδου κ.λπ.

matplotlib.pyplot: Αυτή η βιβλιοθήκη χρησιμοποιείται για τη δημιουργία γραφικών όπως διαγράμματα και γραφήματα.

FigureCanvasTkAgg: Αυτή η υπο-βιβλιοθήκη του matplotlib χρησιμοποιείται για να ενσωματώσει γραφικά που δημιουργήθηκαν με το matplotlib σε παράθυρα tkinter.

requests: Αυτή η βιβλιοθήκη χρησιμοποιείται για την αποστολή HTTP αιτημάτων σε ιστότοπους και τη λήψη των απαντήσεών τους.

BeautifulSoup: Είναι μια βιβλιοθήκη Python που χρησιμοποιείται για την ανάλυση HTML και την εξαγωγή δεδομένων από ιστοσελίδες.

csv: Αυτή η βιβλιοθήκη χρησιμοποιείται για τη δουλειά με αρχεία CSV, συμπεριλαμβανομένης της ανάγνωσης και της εγγραφής δεδομένων.

textstat: Αυτή η βιβλιοθήκη χρησιμοποιείται για τον υπολογισμό διαφόρων μετρήσεων στατιστικής κειμένου, όπως η ευκολία ανάγνωσης του Flesch.

---> 2.Συνάρτηση `scrape_website`

```
11 def scrape_website(url):
12     try:
13         response = requests.get(url) # Αίτημα GET προς τον δοσμένο URL
14         response.raise_for_status() # Έλεγχος για σφάλματα κατά την ανάκτηση του περιεχομένου της σελίδας
15         page_content = response.text # Κείμενο της σελίδας σε μορφή κειμένου
16         soup = BeautifulSoup(page_content, 'html.parser') # Δημιουργία αντικειμένου BeautifulSoup για ανάλυση HTML
17
18         text_content = soup.get_text() # Εξαγωγή κειμένου από την HTML
19
20         # Εξαγωγή μεταδεδομένων (π.χ., τίτλου, περιγραφής)
21         title = soup.title.string.strip() if soup.title else "" # Τίτλος σελίδας, αν υπάρχει
22         description = soup.find('meta', attrs={'name': 'description'}) # Αναζήτηση μεταδεδομένων περιγραφής
23         description = description.get('content').strip() if description else "" # Περιγραφή σελίδας, αν υπάρχει
24
25         # Εύρεση στοιχείων εικόνας και εξαγωγή των συνδέσμων τους
26         image_elements = soup.find_all('img') # Αναζήτηση όλων των στοιχείων εικόνας
27         image_urls = [img.get('src') for img in image_elements] # Εξαγωγή των URLs των εικόνων
28
29         # Εύρεση στοιχείων υπερσυνδέσμων και εξαγωγή των συνδέσμων τους
30         hyperlink_elements = soup.find_all('a') # Αναζήτηση όλων των στοιχείων υπερσυνδέσμων
31         hyperlink_urls = [a.get('href') for a in hyperlink_elements] # Εξαγωγή των URLs των υπερσυνδέσμων
32         num_images = len(image_urls) # Πλήθος εικόνων
33         text_length = len(text_content) # Μήκος κειμένου
34         num_words = len(text_content.split()) # Πλήθος λέξεων
35         unique_words = len(set(text_content.split())) # Πλήθος μοναδικών λέξεων
36         readability_score = flesch_reading_ease(text_content) # Υπολογισμός σκορ διαβασιμότητας
37
38         # Επιστροφή των δεδομένων σε μορφή λεξικού
39         return {
40             'url': url,
41             'text_content': text_content,
42             'title': title,
43             'description': description,
44             'num_images': num_images,
45             'image_urls': image_urls,
46             'text_length': text_length,
47             'num_words': num_words,
48             'unique_words': unique_words,
49             'readability_score': readability_score,
50             'num_hyperlinks': len(hyperlink_urls),
51             'hyperlink_urls': hyperlink_urls
52         }
53     except Exception as e:
54         # Αν προκύψει κάποιο σφάλμα κατά την ανάκτηση δεδομένων, εμφάνιση μηνύματος
55         print(f"Προέκυψε σφάλμα κατά την ανάκτηση δεδομένων από τον ιστότοπο {url}: {e}")
56     return {}
57
```

Η συνάρτηση `scrape_website`, αναλαμβάνει να κάνει web scraping από έναν δοσμένο link από το web

Αίτημα GET: Κάνει ένα αίτημα GET στον δοσμένο URL χρησιμοποιώντας τη βιβλιοθήκη `requests`.

Έλεγχος για σφάλματα: Ελέγχει αν υπάρχουν σφάλματα κατά την ανάκτηση του περιεχομένου της σελίδας, χρησιμοποιώντας τη μέθοδο `raise_for_status()` του αντικειμένου απόκρισης `response`.

Ανάλυση HTML: Χρησιμοποιεί τη βιβλιοθήκη `Beautiful Soup` για να δημιουργήσει ένα αντικείμενο `Beautiful Soup` που θα επιτρέψει την ανάλυση του HTML περιεχομένου της σελίδας.

Εξαγωγή δεδομένων: Από το HTML, εξάγονται τα εξής δεδομένα:

- Κείμενο της σελίδας

- Τίτλος της σελίδας
- Περιγραφή της σελίδας
- Συνδέσμοι εικόνων με τα URLs τους
- Συνδέσμοι υπερσυνδέσμων με τα URLs τους
- Αριθμός εικόνων
- Μήκος κειμένου
- Πλήθος λέξεων
- Πλήθος μοναδικών λέξεων
- Σκορ διαβασιμότητας

Επιστροφή δεδομένων: Τα παραπάνω δεδομένα επιστρέφονται σε μορφή λεξικού.

Αν προκύψει οποιοδήποτε σφάλμα κατά την ανάκτηση δεδομένων, εμφανίζεται ένα μήνυμα σφάλματος και επιστρέφεται ένα κενό λεξικό.

---> 3.Συνάρτηση `save_to_csv`

```

58 def save_to_csv(data, filename):
59     try:
60         with open(filename, 'w', newline='', encoding='utf-8') as output_file:
61             writer = csv.writer(output_file)
62
63             # Εγγραφή αναλύσεων στο CSV
64             writer.writerow(["Analysis", "Value"])
65             writer.writerow(["Number of Images", data.get('num_images', '')])
66             writer.writerow(["Text Length", data.get('text_length', '')])
67             writer.writerow(["Number of Words", data.get('num_words', '')])
68             writer.writerow(["Number of Unique Words", data.get('unique_words', '')])
69             writer.writerow(["Readability Score", data.get('readability_score', '')])
70             writer.writerow(["Number of Hyperlinks", data.get('num_hyperlinks', '')])
71             writer.writerow([]) # Προσθήκη κενής γραμμής για διαχωρισμό
72
73             # Εγγραφή μεταδεδομένων στο CSV
74             writer.writerow(["Metadata", "Value"])
75             for key, value in data.items():
76                 writer.writerow([key, value])
77             messagebox.showinfo("Success", "Website data has been saved to website_data.csv")
78     except Exception as e:
79         messagebox.showerror("Error", f"An error occurred while saving data to CSV: {e}")
80

```

Η συνάρτηση `save_to_csv`, αναλαμβάνει να αποθηκεύσει τα δεδομένα που έχουν συλλεχθεί σε αρχείο CSV

Άνοιγμα αρχείου: Το αρχείο CSV ανοίγει για εγγραφή χρησιμοποιώντας την εντολή `open()` με τις κατάλληλες παραμέτρους.

Δημιουργία writer: Δημιουργείται ένα αντικείμενο `writer` για την εγγραφή δεδομένων στο αρχείο CSV.

Εγγραφή αναλύσεων: Τα δεδομένα ανάλυσης, όπως το πλήθος εικόνων, το μήκος κειμένου κ.λπ., εγγράφονται στο αρχείο CSV.

Εγγραφή μεταδεδομένων: Επιπλέον, εγγράφονται και τα μεταδεδομένα του ιστότοπου, όπως ο τίτλος και η περιγραφή της σελίδας.

Εμφάνιση μηνύματος επιτυχίας: Εάν η εγγραφή ολοκληρωθεί με επιτυχία, εμφανίζεται ένα μήνυμα επιτυχίας με τη χρήση της `messagebox`.

---> 4.Συνάρτηση *display_gui*

```
81 def display_gui():
82     Codeium: Refactor | Explain | Generate Docstring | >X
83     def scrape_and_display():
84         url = url_entry.get() # Λήψη του URL
85         website_data = scrape_website(url) # Ανάκτηση δεδομένων
86         if website_data:
87             save_to_csv(website_data, 'website_data.csv') # Αποθήκευση των δεδομένων σε CSV
88             plot_graph(website_data) # Εμφάνιση γραφήματος των δεδομένων
89             root.destroy() # Κλείσιμο του παραθύρου GUI μετά την αποθήκευση των δεδομένων
90
91     Codeium: Refactor | Explain | Generate Docstring | >X
92     def plot_graph(data):
93         categories = ['Number of Words', 'Number of Unique Words', 'Number of Hyperlinks', 'Number of Images']
94         values = [data.get('num_words', 0), data.get('unique_words', 0), data.get('num_hyperlinks', 0), data.get('num_images', 0)]
95
96         plt.figure(figsize=(8, 5))
97         bars = plt.bar(categories, values, color=['blue', 'green', 'orange', 'red'])
98
99         # Προσθήκη αριθμησης σε κάθε μπάρα
100         for bar in bars:
101             yval = bar.get_height()
102             plt.text(bar.get_x() + bar.get_width()/2.0, yval, int(yval), va='bottom')
103
104         plt.xlabel('Categories')
105         plt.ylabel('Count')
106         plt.title('Website Data')
107         plt.xticks(rotation=45) #Για καλύτερη αναγνωσιμότητα
108         plt.tight_layout() #Για να μην γίνεται περικοπή στις ετικέτες
109         plt.show()
110
111     root = tk.Tk() # Δημιουργία του κύριου GUI
112     root.title("Website Scraper")
113
114     style = ttk.Style()
115     style.theme_use('clam') # Ορισμός του στυλ σε 'clam'
116
117     # Πλαίσιο
118     main_frame = ttk.Frame(root)
119     main_frame.pack(padx=20, pady=20)
120
121     # Καταχώρηση URL
122     url_label = ttk.Label(main_frame, text="Enter the URL of the website:")
123     url_label.grid(row=0, column=0, padx=10, pady=10, sticky="w")
124     url_entry = ttk.Entry(main_frame, width=50)
125     url_entry.grid(row=0, column=1, padx=10, pady=10)
126
127     scrape_button = ttk.Button(main_frame, text="Retrieve Website Data", command=scrape_and_display)
128     scrape_button.grid(row=0, column=2, padx=10, pady=10)
129
130     root.mainloop() # Έναρξη εκτέλεσης του GUI
131
132 if __name__ == "__main__":
133     display_gui() # Κλήση της συνάρτησης για την εμφάνιση του GUI
```

Η συνάρτηση `display_gui` αναλαμβάνει τη δημιουργία ενός γραφικού περιβάλλοντος χρήστη (GUI) για τον χρήστη που θέλει να χρησιμοποιήσει την εφαρμογή για την ανάκτηση και αποθήκευση δεδομένων από το web.

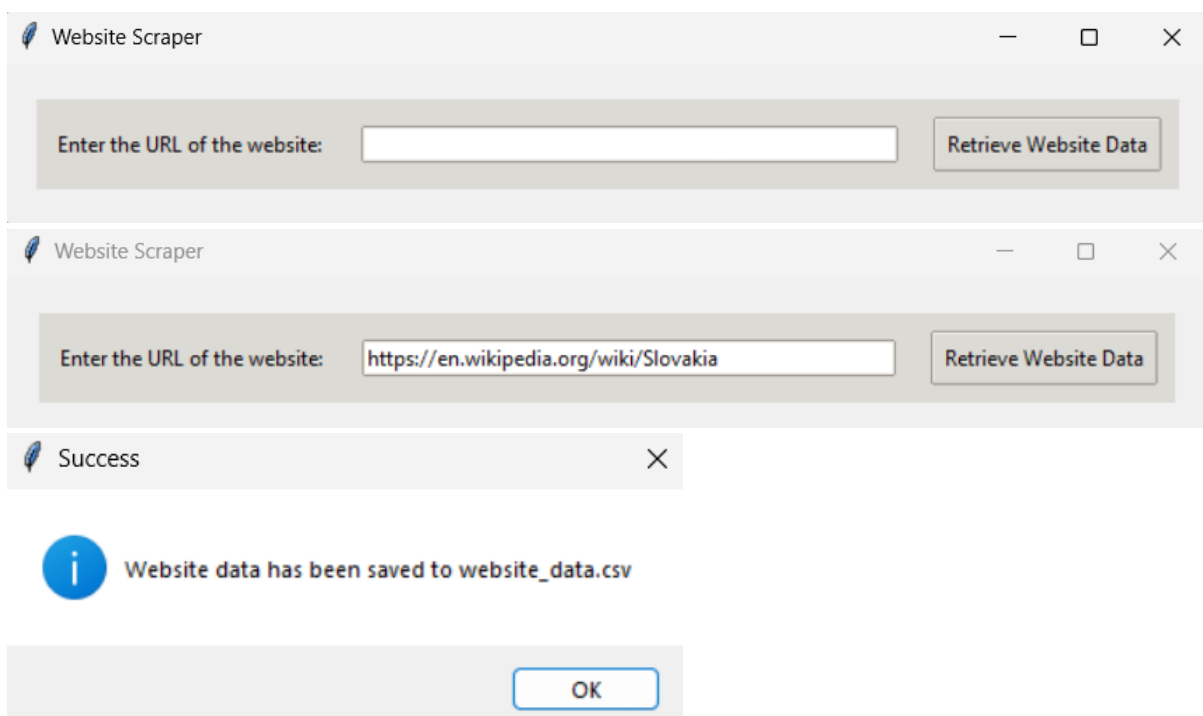
Συνάρτηση `scrape_and_display`: Αυτή η συνάρτηση καλείται όταν ο χρήστης πατάει το κουμπί "Ανάκτηση Δεδομένων Ιστότοπου". Αρχικά, λαμβάνει το URL που έχει εισαγάγει ο χρήστης από το πλαίσιο κειμένου. Έπειτα, χρησιμοποιεί τη συνάρτηση `scrape_website` για να ανακτήσει τα δεδομένα από τον ιστότοπο. Αν τα δεδομένα ανακτηθούν με επιτυχία, τότε αποθηκεύονται σε ένα αρχείο CSV με τη χρήση της συνάρτησης `save_to_csv` και εμφανίζεται ένα γράφημα με τη χρήση της συνάρτησης `plot_graph`. Τέλος, το παράθυρο GUI κλείνει με την εντολή `root.destroy()`.

Συνάρτηση `plot_graph`: Αυτή η συνάρτηση δημιουργεί ένα γράφημα που δείχνει τον αριθμό των λέξεων, μοναδικών λέξεων, υπερσυνδέσμων και εικόνων που βρέθηκαν στον ιστότοπο. Τα δεδομένα που χρησιμοποιούνται για τη

δημιουργία του γραφήματος προέρχονται από την παράμετρο `data`, η οποία περιέχει τα αποτελέσματα της ανάκτησης του ιστότοπου.

Δημιουργία GUI: Δημιουργείται το κύριο παράθυρο GUI χρησιμοποιώντας το `tkinter`. Το παράθυρο έχει τον τίτλο "Καταγραφέας Ιστότοπου". Ορίζεται το στυλ των επιλεγμένων στοιχείων ως "clam". Στη συνέχεια, προστίθενται τα στοιχεία του πλαισίου όπως το πλαίσιο κειμένου για το URL, το πλαίσιο εισαγωγής για το URL και το κουμπί για την έναρξη της ανάκτησης δεδομένων. Το GUI τερματίζει την εκτέλεσή του μόλις το παράθυρο κλείσει ή ο χρήστης κλείσει την εφαρμογή.

---> 5. Το GUI



---> 6.Το γράφημα

